Deep Q-Learning

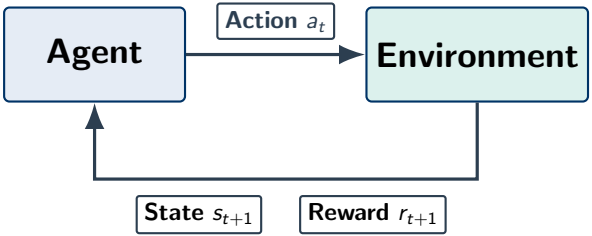
for Optimal Execution in Financial Markets

Matthew Millward
Durham University
Supervised by Marija Zamaklar

11 February 2026

What is Reinforcement Learning?

An Introduction



Goal: Learn a policy π that maximises cumulative reward over time.

Supervised Learning

Training on **labelled data**. A "teacher" provides the correct answer for every input. Used for classification or perception problems.

Reinforcement Learning

Learns through **experience**, discovering strategies that maximise long-term reward. Used for sequential decision-making problems [1].

A Running Example

A 3×3 grid world we'll revisit throughout

A	s_2	s_3
s_4	s_5	s_6
X	s_8	Goal

Reward Structure

- Reaching the goal: **+10**
- Landing on X: **-10**
- Each step: **-1**
- Actions: $\{\uparrow, \downarrow, \leftarrow, \rightarrow\}$
- Discount factor: $\gamma = 1$

Note: The step penalty encourages the agent to find the shortest path while avoiding the penalty zone.

The Q-Table

A lookup table storing action-value estimates

State	Up	Down	Left	Right
s_1	**0**	0	0	0
s_2	0	0	0	0
s_3	0	0	0	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	0	0	0	0

$\hat{Q}(s_1, \text{Up}) = 0 \leftarrow$ current estimate

9 states $\times$ 4 actions $=$ 36 entries

Action-Value Function (Q-Value)

The Q-value measures how good it is to take a particular action in a given state, averaged over all possible paths the agent might follow afterward.

$$Q(s,a) = \mathbb{E}\Big[\underbrace{r_{t+1} + \gamma\, r_{t+2} + \gamma^2 r_{t+3} + \cdots}_{\text{Total discounted reward}} \mid s, a\Big]$$

We can decompose this recursively [1]:

$$Q(s,a) = \mathbb{E}\Big[\underbrace{r_{t+1}}_{\text{Immediate reward}} + \underbrace{\gamma \max_{a'} Q(s_{t+1}, a')}_{\text{Discounted next state}} \mid s, a\Big]$$

The Q-Learning Update

Walking through one step

Example: One Step

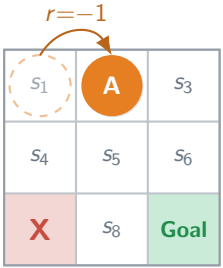
1. **Observe** a transition
2. **Estimate** the target
3. **Compute** the error
4. **Update** $\hat{Q}$ toward target

↻ Repeat from new state s'

Update Rule [2]

$$\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \alpha \cdot \overbrace{\Big[\underbrace{r + \gamma \max_{a'} \hat{Q}(s',a')}_{\text{Target}} - \underbrace{\hat{Q}(s,a)}_{\text{Old estimate}}\Big]}^{\text{Error}}$$

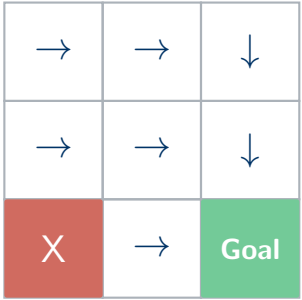
Learning rate: $\alpha = 0.05$ Discount factor: $\gamma = 1$



After Learning: The Converged Q-Table

5,000 episodes later — optimal policy discovered

State	Up	Down	Left	Right
S_1	6.0	**7.0**	6.0	**7.0**
S_2	7.0	**8.0**	6.0	**8.0**
S_3	8.0	**9.0**	7.0	8.0
S_4	6.0	**-10.0**	7.0	**8.0**
S_5	7.0	**9.0**	7.0	**9.0**
S_6	8.0	**10.0**	8.0	**9.0**
S_7 (X)	0.0	0.0	0.0	0.0
S_8	8.0	9.0	**-10.0**	**10.0**
S_9 (Goal)	0.0	0.0	0.0	0.0

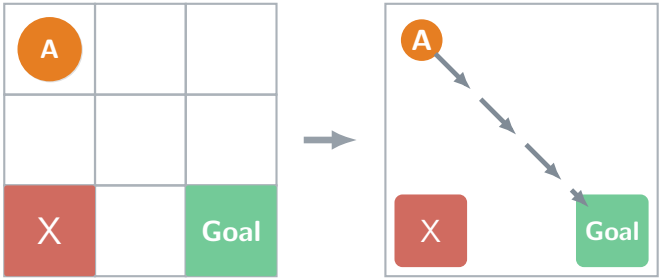


Acting greedily under this policy

Agent discovered optimal paths to the goal and learned to avoid the penalty state through trial and error, *without ever seeing the grid layout*.

Why Tabular Q-Learning Breaks Down

While tabular methods work great for small problems, they don't scale



Discrete — table works **Continuous — no table**

The Scale Problem

- Chess: $\sim 10^{43}$ states
- Continuous environments $\rightarrow$ infinite states

No Generalisation

Each Q-value is learned independently; similar states share no info.

What We Need

A function that approximates Q and generalises $\rightarrow$ **Neural networks**

Deep Q-Networks

Replacing the Q-table with a neural network

Function Approximation

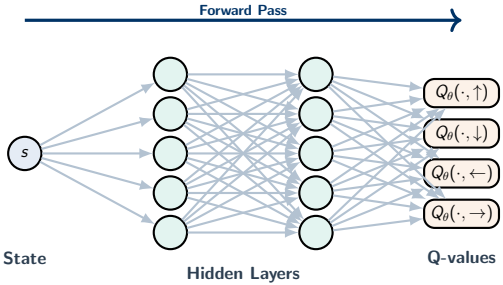
Instead of a table, use a **neural network** with parameters θ to approximate [3]:

$$Q_\theta(s, a) \approx Q(s, a)$$

Each layer applies a linear map followed by a non-linear transformation to the input $\boldsymbol{x}$:

$$\boldsymbol{a} = \underbrace{\sigma}_{\text{Non-linear function}}\Big(\underbrace{W\boldsymbol{x} + \boldsymbol{b}}_{\text{Linear map}}\Big)$$

The non-linear function of choice is typically ReLU: $\sigma(z) = \max\{0, z\}$.



Objective: Learn parameters $\theta = (W^{(1)}, \boldsymbol{b}^{(1)}, \dots)$ that best approximate $Q(s, a)$.

Training the DQN

Learning the network parameters

Learning Process

1. **Observe** a transition
2. **Build** the target

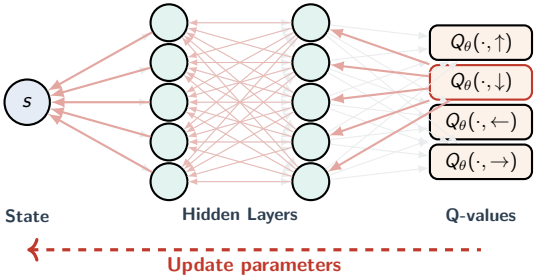
   $$y = r + \gamma \max_{a'} Q_\theta(s', a')$$

3. **Compute** the loss

   $$\mathcal{L}(\theta) = \frac{1}{2}(y - Q_\theta(s, a))^2$$

4. **Adjust** θ to reduce loss

↻ Repeat from new state s'



Note: *Parameters are adjusted using backpropagation and gradient descent.*

Stabilising DQN: Experience Replay

Breaking correlation in training data

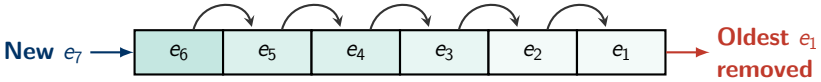
The Problem

Sequential experiences are very similar to each other. Learning from correlated data produces high variance when updating parameters, making training unstable.

The Solution

Store past experiences in a **replay buffer** [3]. Sample random batches of experiences for training. This breaks correlation and reduces variance in updates.

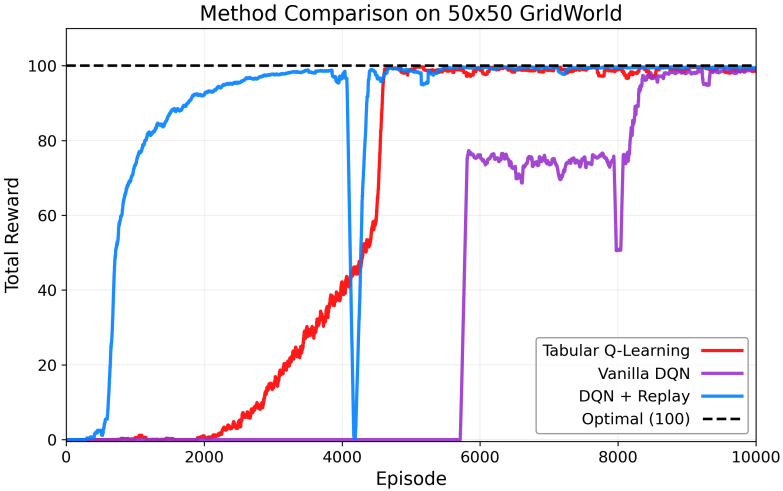
Replay Buffer



Each experience $e_i = (s, a, r, s')$ stored and sampled randomly for training

DQN Performance

Comparison of tabular learning vs DQN adaptations



Optimal Execution: The Problem

How should large investors sell their stock?

The Challenge

Large investors often need to sell huge quantities of stock.

The Trade-off:

- **Sell fast** $\rightarrow$ You run out of buyers and push prices down.
- **Sell slow** $\rightarrow$ Prices may move against you while you wait.

Classical Solution: Almgren-Chriss [4]

Use calculus to find a fixed selling schedule.

Why it fails:

- Assumes the number of buyers stays constant (unrealistic).
- Schedule is rigid and predictable.
- Other traders can spot and exploit the pattern.

Optimal Execution: The Solution

Replacing rigid schedules with adaptive agents

Reinforcement Learning Setup

We can instead frame this as a reinforcement learning problem:

- **State:** Inventory remaining, time elapsed, market conditions.
- **Action:** Trading speed (How many shares to sell *now*).
- **Reward:** Minimise execution cost (sell at better prices $\rightarrow$ higher reward).

Why Deep Q-Learning?

1. **Model-free:**
   - No assumptions about market dynamics are needed.
2. **Learns non-linear strategies [5]:**
   - Pauses selling when buyers disappear.
   - Detects and evades adversarial agents.
   - Adapts the selling schedule in real-time.

Conclusions & Future Work

Conclusion:

- **Reinforcement Learning:** Learn by trial and error to maximise reward.
- **Tabular Q-Learning:** Works for small problems, doesn't scale.
- **Deep Q-Networks:** Introducing neural networks & experience replay.
- **Optimal Execution:** DQN learns adaptive trading strategies.

Future Work:

- Testing against predatory agents.
- Evaluation on real limit order book data.

Thank you — questions?

References

1. Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction*. MIT Press.
2. Watkins, C.J.C.H. & Dayan, P. (1992). Q-learning. *Machine Learning*.
3. Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*.
4. Almgren, R. & Chriss, N. (2000). Optimal execution of portfolio transactions. *Journal of Risk*.
5. Ning, B., Lin, F.H.T., Jaimungal, S. (2021). Double Deep Q-Learning for Optimal Execution. *Applied Mathematical Finance*, 28(4), 361-380.

Code Repository